

oom-watch — Deployment Guide

One-shot install + verify for the OOM-Protect monitoring daemon

OOM-Protect maintainers

2026-04-30

oom-watch — Deployment Guide

- 1. The one command
- 2. Prerequisites
 - 2.1 ALT Linux specifics
 - 2.2 Cross-UID repository note
- 3. What the script does, step by step
 - 3.0 Optional `--pull`
 - 3.1 Pre-flight
 - 3.2 Build + install
 - 3.2a Post-install path checks
 - 3.2b Validate `/etc/oom-watch/config.json`
 - 3.3 daemon-reload + enable + restart
 - 3.4 Wait for `active` state (timeout 30 s)
 - 3.5 Journal sanity check
 - 3.6 Wait for first report (timeout 60 s)
 - 3.7 Summary
- 4. Flags
- 5. Exit codes
- 6. Idempotency and re-running
- 6a. The diagnostic bundler — `make oomwatch-diagnose`
 - What the bundle captures
 - Flags
 - Privacy considerations
 - Typical workflow
- 6b. A real deployment session, end to end
 - What “calm host” means
- 7. Failure modes
- 8. Reverse and uninstall
- 9. After deploy: live operations
- 10. Constitution alignment (Article I)
- See also

oom-watch — Deployment Guide

This guide is the canonical, end-to-end procedure for getting `oom-watch` running on a host. It documents the deployer script (`oom-watch/scripts/install-and-verify.sh`) and the `make oom-watch-deploy` target that wraps it.

If you are debugging a failed deployment, jump straight to **§7 Failure modes** — every documented symptom maps to a specific assertion the script makes, and the EXIT trap dumps the corresponding diagnostic automatically.

1. The one command

On a target host with the OOM-Protect repo checked out, atop installed, and Go $\geq$ 1.22 available:

```
sudo make oomwatch-deploy
```

...or, when already root (e.g. logged in via `su -`):

```
make oomwatch-deploy
```

That is the entirety of a fresh deploy. The Makefile target invokes `oom-watch/scripts/install-and-verify.sh`, which performs every step described below in order, asserts each one, and aborts with a full diagnostic dump if any check fails.

If your repo is checked out somewhere other than the working directory, run the script directly:

```
sudo bash /path/to/OOM-Protect/oom-watch/scripts/install-and-verify.sh
```

2. Prerequisites

The script will refuse to proceed unless every prerequisite is satisfied. There are no silent skips.

Prerequisite	Why	Install hint per distro
Root or sudo	Writes to <code>/usr/local/sbin</code> , <code>/etc/oom-watch</code> , <code>/etc/systemd/system</code> , <code>/var/log/oom-watch</code> , <code>/var/lib/oom-watch</code>	ALT: <code>su -</code> (root often not in sudoers — <code>make</code> auto-detects EUID 0). Debian/Ubuntu/RHEL: <code>sudo ...</code> works
<code>atop</code> on PATH	The daemon's only data source. Without atop the daemon refuses to start; this is intentional (no silent /proc fallbacks).	ALT: <code>apt-get install atop</code> . Debian/Ubuntu: <code>apt install atop</code> . RHEL/Fedora: <code>dnf install atop</code> . SUSE: <code>zypper install atop</code> . Verified version: 2.12.1 (Sep 2025)
<code>systemctl</code>	The unit is managed by systemd.	Included with systemd; tested on systemd 258 (ALT 11)
Linux kernel ≥ 5.15	Pressure Stall Information (PSI) on <code>/proc/pressure/memory</code> is the strongest leading indicator of imminent OOM and a primary signal in the threshold engine.	All recent distros
cgroup v2 unified hierarchy	The daemon reads <code>/sys/fs/cgroup/user.slice/user-<code><uid></code>.slice/memory.*</code> for the user-slice section of reports. cgroup v1 hosts produce empty for that section, otherwise work fine.	Default on most modern distros
Go ≥ 1.22	Only required if <code>oom-watch/oomwatch</code> has not already been built.	ALT: <code>apt-get install golang</code> . Debian/Ubuntu: <code>apt install golang-go</code> . Tested with Go 1.26.2
<code>make</code>	Drives the install target.	usually pre-installed

The script does **not** install atop or Go for you. Fixing prerequisites is intentionally a separate step from deployment so you can review what you're installing.

2.1 ALT Linux specifics

This project was developed and field-tested on **ALT Linux 11 “Salvia” (Sisyphus branch)**, kernel 6.12.61, systemd 258, atop 2.12.1. A few ALT-specific behaviours the deployer accommodates automatically:

- **Root is not in sudoers** by default on ALT (a security choice — root has no need to authenticate to itself). Earlier versions of the Makefile blindly prepended `sudo` and bricked under `su -`. The current Makefile evaluates `SUDO := $(shell test "$(id -u)" = "0" && echo "" || echo "sudo")` once at parse time, so the same `make oomwatch-deploy` works as both root (skips sudo) and a sudo-capable user (prepends it). Override with `make ... SUDO="doas"` for distros using doas.
- **systemd 258 directive forms.** `StartLimitIntervalSec` must live in `[Unit]` (older systemd tolerated `[Service]`); `ProtectControlGroups=read-only` is rejected (only `yes / no` accepted on this release). The shipped unit file is correct for systemd ≥ 230 and the comments call out the symptoms (“Unknown key ... in section [Service]”) if you ever see them in the journal.
- **PSI present**, kernel 6.12 emits `/proc/pressure/{memory,cpu,io}` cleanly.

2.2 Cross-UID repository note

If the OOM-Protect checkout lives on a filesystem owned by a UID different from the one running the deployer (e.g. an external mount owned by your user, deployer run as root), Go’s default VCS-stamping in `go build` triggers git’s “dubious ownership” safety feature and the build fails with `error obtaining VCS status: exit status 128`.

The deployer and Makefile pass `-buildvcs=false` to every `go build`, and `challenges/lib.sh` exports `GOFLAGS=-buildvcs=false` for every Challenge that builds a helper. The flag is zero-cost (we don’t read the VCS stamp from the binary) and removes the entire failure class. You don’t need to do anything; this section just explains why the flag is there if you encounter it.

If you genuinely want VCS stamping, set `git config --global --add safe.directory /path/to/OOM-Protect` first and remove `-buildvcs=false` overrides — but there is no functional benefit.

3. What the script does, step by step

The script is `oom-watch/scripts/install-and-verify.sh`. Each numbered step here corresponds to a header section in the script’s stderr output.

3.0 Optional `--pull`

If invoked with `--pull`, runs `git pull --ff-only` in the repo so that the binary you build, the example config you install, and the unit file all come from current `main`. **Off by default** because pulling on a dirty tree may conflict; turn it on for fully-automated update-and-redeploy.

3.1 Pre-flight

- Re-execs under sudo when not root (or fails loudly if sudo is missing on a non-root host).
- Verifies `atop` is on PATH and prints its version.
- Verifies `systemctl` is reachable.

3.2 Build + install

- Builds `oom-watch/oomwatch` if missing (or always, with `--rebuild`). Skipped entirely with `--no-install`.
- Runs `make oomwatch-install`, which is itself idempotent:
 - copies the binary to `/usr/local/sbin/oomwatch`
 - creates `/etc/oom-watch/`, `/var/log/oom-watch/`, `/var/lib/oom-watch/`
 - copies `oom-watch/config/oom-watch.example.json` to `/etc/oom-watch/config.json` **only if the file does not already exist** (preserves customised configs)
 - copies `oom-watch/systemd/oom-watch.service` to `/etc/systemd/system/`

3.2a Post-install path checks

Asserts every artefact landed where expected: binary executable, config file present, unit file present. Catches a regression in `make oomwatch-install` before any service ever tries to start.

3.2b Validate `/etc/oom-watch/config.json`

Runs `oomwatch -dry-run` against the **installed** config. This is the anti-bluff guard against a class of bug where the daemon enters the systemd start path, fails configuration parsing, and dies at exit code 2 with no human-readable diagnostic.

If validation fails, the script auto-remediates:

1. The broken file is moved aside to `/etc/oom-watch/config.json.broken.<timestamp>`.
2. The shipped example is copied in as the new config.
3. The new config is re-validated.

If even the fresh example fails validation, the repository itself is broken and the script exits fatal. The backup path is printed; if you had custom thresholds, you can re-apply them by diffing the backup against the current file.

3.3 daemon-reload + enable + restart

Reload the systemd manager configuration, enable the unit if not already enabled, and restart the unit. Restart (vs. start) gives the verifier a known starting state.

3.4 Wait for `active` state (timeout 30 s)

Polls `systemctl is-active oom-watch.service` once per second:

- `active` → proceed.

- `failed` → exit 1 immediately (no point in waiting).
- anything else → keep polling.

Times out at 30 s. The systemd unit is `Type=simple`, so it should reach `active` within milliseconds; staying in `activating` for 30 s indicates the binary is crashing on startup. The EXIT trap will then dump `systemctl status` and the journal so you can see why.

3.5 Journal sanity check

Asserts `journalctl -u oom-watch.service --since "2 minutes ago"` contains the line `atop located`. The daemon prints this once it has resolved `atop` on its restricted (sandboxed) PATH. A daemon that bounced because the systemd hardening blocked something would otherwise show `active` but not have run; this check rules that out.

3.6 Wait for first report (timeout 60 s)

Polls `/var/log/oom-watch/reports/` for any `.md` file. A genuinely calm host may not produce ANY automatic report — quiet IS the goal — so if the wait expires, the script forces a `-one-shot` diagnostic so we have proof on disk that the *report-writing* path also works.

3.7 Summary

Prints unit state, binary mode and ownership, paths, and the latest report file name and size, plus the live-tail command so you know how to follow the daemon going forward.

4. Flags

```
sudo bash oom-watch/scripts/install-and-verify.sh [FLAGS]
```

Flag	Effect
<code>--pull</code>	<code>git pull --ff-only</code> the repo before building. Off by default.
<code>--no-install</code>	Skip step 3.2 entirely. Use to verify a running unit after a manual restart.
<code>--rebuild</code>	Force a fresh <code>go build</code> even if the binary already exists.
<code>--quiet</code>	Suppress per-step OK / informational prints. Failure diagnostics are always printed.
<code>--help</code> , <code>-h</code>	Print the script's header documentation and exit.

5. Exit codes

Code	Meaning
0	Success. The daemon is installed, active, and producing reports.
1	Any pre-flight, install, validation, or runtime check failed. The EXIT trap will have dumped <code>systemctl status</code> , the last 50 journal lines, the config file, the unit file, and a listing of the report directory.
64	Usage error (unknown flag).

6. Idempotency and re-running

The script is **idempotent by design**. Re-running on a host where `oom-watch` is already deployed is safe; it will:

- Skip the binary build if `oom-watch/oomwatch` is current (override with `--rebuild`).
- Skip `enable` if the unit is already enabled.
- **Always** restart the unit (gives a clean starting state for verification).
- **Always** re-validate the installed config and auto-remediate if broken.

Re-running is also the recommended remediation when something has drifted on the host — it converges the host to the deployed state.

6a. The diagnostic bundler — `make oomwatch-diagnose`

When a deploy fails and you want every relevant piece of state in one paste-able file, run:

```
sudo make oomwatch-diagnose
```

...or, when already root:

```
make oomwatch-diagnose
```

The script (`oom-watch/scripts/diagnose.sh`) writes a single `/tmp/oomwatch-diagnose-
<timestamp>.log` containing 16 numbered sections — every command's output is captured, even when the command itself fails (the script intentionally runs with `set -u` but **without** `set -e`, so one section's failure doesn't abort the others; failures are themselves diagnostic evidence).

What the bundle captures

§	Section	Why
0	Timestamp + host identity	Pin the snapshot in time
1	<code>uname -a</code> + <code>/etc/os-release</code>	Distro / kernel context
2	Tool versions (atop, systemctl, go, make, git)	Rule out version skew
3	Repo state: HEAD commit, dirty files, remotes, branch tracking	Confirm you're on the expected commit; catch a <code>git pull</code> that no-op'd because of conflicts
4	First 5 lines + SHA256 of the SHIPPED <code>oom-watch.example.json</code> in the repo	Proves whether the example in your tree is the fixed version
5	<code>ls -la</code> of every install path (binary, config, unit, log dirs, lib dir)	Reveals partial installs
6	Full <code>cat</code> + SHA256 of <code>/etc/oom-watch/config.json</code>	The single most important artefact when the daemon fails to start
7	<code>oomwatch -dry-run</code> against the installed config + rc	The validator's verdict in one line
8	<code>systemctl is-enabled / is-active / is-failed / status -n 50</code>	Unit lifecycle state
9	Full <code>cat</code> of <code>/etc/systemd/system/oom-watch.service</code>	Catches sandbox / capability drift
10	<code>journalctl -u oom-watch.service -n 200 --no-pager</code>	The daemon's actual reason for exiting
11	<code>/var/log/oom-watch/reports/</code> listing + count	Has the daemon ever produced a report?
12	A live <code>atop -PMEM,PSI,CPL 1 2</code> sample	Sanity check that atop IT-SELF works on this host
13		

§	Section	Why
	A full <code>make oomwatch-deploy</code> run with all step headers and any EXIT-trap dump	Reproduces the failure under instrumentation
14	Post-deploy <code>is-active + status -n 30</code>	Final unit state after the deploy attempt
15	Post-deploy <code>journalctl -n 50</code>	What the daemon said during the most recent attempt

Flags

Flag	Effect
<code>--no-deploy</code>	Skip §13–15 (don't run <code>make oomwatch-deploy</code>); snapshot only the current state. Use when you want to capture context without disturbing a stable system.
<code>--help</code> , <code>-h</code>	Print the script's header documentation and exit.

Pass flags via `make` like so:

```
make oomwatch-diagnose DIAGNOSE_FLAGS="--no-deploy"
```

Privacy considerations

The bundle contains:

- Your hostname.
- Process names from `atop -PMEM,PSI,CPL` (typically only top-level command names, no arguments).
- The last 200 journal lines for `oom-watch.service` (daemon-internal logs only — not other unit logs).

It does **not** contain:

- Other users' processes' command lines.
- Other systemd units' journal entries.
- Disk contents beyond the named config / unit files.

The log file is written to `/tmp/`, which most distros clear on reboot. The repository's `.gitignore` excludes `oomwatch-diagnose-*.log` and `*.diagnose.log` so a copy accidentally placed in the working tree won't be committed.

Typical workflow

```
# 1. Reproduce the failure under the diagnostic harness:  
sudo make oomwatch-diagnose  
  
# 2. Note the printed log path (e.g. /tmp/oomwatch-diagnose-20260430-163342.log).  
  
# 3. Read it yourself first – most issues are obvious from the $6 cat  
#    (config) and $7 dry-run rc + $10 journal.  
  
# 4. If you need help, paste the full log into your support channel.
```

6b. A real deployment session, end to end

This section shows the deployment as it actually plays out on a fresh host, with sample output at each step. Use it as a sanity check while watching your own deploy.

```

# 1. Prerequisites (one-time per host)
$ apt-get install atop                                     # ALT Linux: atop 2.12.1+
$ which atop && atop -V | head -1
/usr/bin/atop
Version: 2.12.1 - 2025/09/23 19:13:15

# 2. Clone or update the repo (typical: ~/OOM-Protect)
$ cd /path/to/OOM-Protect
$ git pull --ff-only
Already up to date.

# 3. Deploy as root (or via sudo)
$ make oomwatch-deploy
== 1. Pre-flight ==
[deploy] OK atop present: /usr/bin/atop (2.12.1)
[deploy] OK systemd present: systemd 258 (258.5-alt1)

== 2. Build + install ==
[deploy] OK binary built: ../oom-watch/oomwatch
[deploy] running 'make oomwatch-install' (idempotent)
installed: /etc/oom-watch/config.json (from example)
installed.Enable with: systemctl enable --now oom-watch.service
[deploy] OK make oomwatch-install completed
[deploy] OK post-install paths verified: binary, config, unit

== 2b. Validate /etc/oom-watch/config.json ==
[deploy] OK /etc/oom-watch/config.json passes -dry-run

== 3. systemd: daemon-reload + reset-failed + enable + restart ==
[deploy] OK reset-failed cleared (unit can start fresh)
[deploy] service already enabled
[deploy] OK service restarted (clean slate for verification)

== 4. Waiting for unit to reach 'active' (timeout 30 s) ==
[deploy] OK unit is active

== 5. Recent journal ==
... msg="oom-watch starting" version=dev interval_s=10 report_dir=/var/log/oom-watch/reports
... msg="atop located" path=/usr/bin/atop
[deploy] OK journal contains 'atop located' (daemon reached the sample loop)

== 6. Waiting for first report (timeout 60 s) ==
[deploy] no automatic report yet (host appears calm); requesting -one-shot diagnostic
... msg="one-shot report written" path=/var/log/oom-watch/reports/2026-04-30T16-57-17Z-notice.md severity=NOTICE
[deploy] OK first report on disk: ../2026-04-30T16-57-17Z-notice.md (23960 bytes)

== Summary ==
Unit:      active
Binary:    -rwxr-xr-x root 4120181 /usr/local/sbin/oomwatch
Config:    /etc/oom-watch/config.json
Unit file: /etc/systemd/system/oom-watch.service
Reports dir: /var/log/oom-watch/reports/
Last report: ../2026-04-30T16-57-17Z-notice.md (23960 bytes)

```

```
Follow logs: journalctl -fu oom-watch.service
[deploy] OK oom-watch is installed, active, and producing reports.

# 4. Verify under real pressure (optional but recommended once per host)
$ make challenges
... [seven Challenges run] ...
challenges passed: 7 / 7
```

Total time: ~30 seconds for the deploy itself, plus ~60 seconds for `make challenges`. After that, the daemon is running, will sample every 10 seconds, and will write reports if any threshold breaches.

What “calm host” means

If your host is comfortably under all thresholds at deploy time (typical idle state: 30-50 % memory used, swap untouched, PSI near zero, load < 1 per CPU), the deployer’s step 6 will see no automatic report after 60 s and **force a -one-shot diagnostic** so we have proof on disk that the report-writing path works. That is **not a failure** — quiet IS the goal of this daemon. The forced one-shot lands as `<timestamp>-notice.md` regardless.

After deployment, the daemon stays quiet until something genuinely starts going wrong. Tail `journalctl -fu oom-watch.service` to confirm it is iterating without errors; the absence of new reports is the success signal.

7. Failure modes

Every failure surfaces a specific error from the script (above the EXIT-trap diagnostic dump). The diagnostic dump always includes `systemctl status`, journal tail, config, unit file, and report directory listing.

Symptom	Most likely cause	What to do
<code>atop</code> is not installed	<code>atop</code> binary not on PATH	Install <code>atop</code> , retry.
<code>systemctl</code> not found	Host is not <code>systemd</code>	This script is <code>systemd</code> -only; for non- <code>systemd</code> hosts run <code>oomwatch -one-shot</code> from <code>cron</code> or your supervisor of choice.
Go toolchain required to build <code>oomwatch</code>	No prebuilt binary AND no Go on host	Either install Go and retry, or copy a prebuilt <code>oom-watch/oomwatch</code> from another host.
INSTALLED CONFIG IS INVALID	<code>/etc/oom-watch/config.json</code> fails <code>-dry-run</code>	The script auto-remediates: backs up to <code>config.json.broken.<ts></code> , replaces with shipped example. If the validator output mentions an unknown field, the repo's example may also be broken — file an issue.
unit went to 'failed' state	The daemon process exits non-zero before the verifier polls	The EXIT-trap dump shows the precise cause: usually a sandbox path the daemon needs is blocked, or the binary is built for a different architecture.
unit did not reach 'active' within 30 s	<code>Type=simple</code> daemon stuck activating	Same as above — read the dumped journal. Common cause: <code>Restart=on-failure</code> is bouncing because the binary is crashing immediately.
journal does not contain 'atop located'	Daemon got past unit start but never reached the sample loop	<code>atop</code> probably failed to spawn. Look at the dumped journal for <code>atop</code> -related errors.
even <code>-one-shot</code> did not produce a report	<code>atop</code> emits zero samples or <code>report-dir</code> is unwritable	Check the report dir's mode and the daemon's <code>User=</code> in the unit file (default <code>root</code>).

8. Reverse and uninstall

There is no first-class uninstall yet (TODO). To manually uninstall:

```

sudo systemctl disable --now oom-watch.service
sudo rm /etc/systemd/system/oom-watch.service
sudo systemctl daemon-reload
sudo rm /usr/local/sbin/oomwatch
sudo rm -rf /etc/oom-watch /var/log/oom-watch /var/lib/oom-watch

```

Note: removing `/var/log/oom-watch` deletes any past forensic reports. If you want to keep them, move the directory aside first.

9. After deploy: live operations

```

# Tail the daemon log:
journalctl -fu oom-watch.service

# List recent reports:
ls -lt /var/log/oom-watch/reports/ | head -20

# Open the most recent report (any severity):
ls -t /var/log/oom-watch/reports/*.md | head -1 | xargs less

# Open the most recent CRITICAL:
ls -t /var/log/oom-watch/reports/*-critical.md 2>/dev/null | head -1 | xargs less

# Show effective config (defaults overlaid with the file):
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -print-config

# Force a diagnostic report right now:
sudo /usr/local/sbin/oomwatch -config /etc/oom-watch/config.json -one-shot

# Clean up old reports (cron candidate):
sudo find /var/log/oom-watch/reports -name '*.md' -mtime +30 -delete

```

10. Constitution alignment (Article I)

This deployer is itself an anti-bluff artefact. Each numbered step asserts an observable outcome — file on disk, exit code, systemd unit state, journal substring, report file size — and exits non-zero on the first failed assertion. There are no `|| true`s, no `t.Skip`-equivalents, and no silent successes. The EXIT-trap diagnostic dump means a failure always surfaces enough context for the operator to act without re-running anything.

The corresponding regression Challenge is `challenges/challenge-config-validation.sh`, which asserts the shipped `oom-watch.example.json` itself passes `-dry-run`. A future commit that re-introduces an unknown field in the example config would fail that Challenge, not your fresh install.

See also

- `manuals/oom-watch-runbook.md` — incident-response playbook. Every issue we have hit in production or during deployment is documented there with one-command diagnosis and exact remediation. Read it once now; bookmark for 3 a.m.
- `manuals/oom-watch-manual.md` — full daemon documentation (config keys, severity ladder, threshold tuning, report anatomy).
- `reports/oom-watch-architecture.md` — design decisions, including why the deployer validates with `-dry-run` before enabling and why the systemd sandbox is deliberately moderate.
- `Constitution.md` — the project charter and Article I (anti-bluff testing).